

Components (Functional & Class Components)

Question 1: What are the components in React? Explain the difference between functional components and class components.

Answer: Components are the building blocks of a React application. They are independent, reusable UI components that combine layout (JSX) and logic.

- **Functional Components:** These are plain JavaScript functions that accept props as an argument and return JSX. They use **Hooks** (useState) to manage internal state and side effects. They are the modern standard in React development.
- **Class Components:** These are ES6 classes that extend React.Component. They require a render() method that returns JSX and manages state using this.state and this.setState(). They also rely on traditional lifecycle methods (componentDidMount).

Question 2: How do you pass data to a component using props?

Answer: Props (properties) are passed to a component just like HTML attributes. The parent component defines the attributes, and the child component receives them grouped together inside a single JavaScript object.

```
// Parent component passes data
<Greeting name="John" />
```

```
// Child component receives data
function Greeting(props) {
  return <h1>{props.name}</h1>;
}
```

Question 3: What is the role of render() in class components?

Answer: The render() method is the only mandatory method in a React class component. Its job is to look at the component's current props and state and return the JSX template that defines what should actually be painted onto the screen. React triggers this method automatically every time the component's state changes.

LAB EXERCISE

Task 1: Create a functional component Greeting that accepts a name as a prop and displays "Hello, [name]!".

```
import React from 'react';
function Greeting(props) {
  return (
    <div>
      <h1>Hello, {props.name}!</h1>
    </div>
  );
}
export default Greeting;
```

Task 2: Create a class component WelcomeMessage that displays "Welcome to React!" and a render() method.

```
import React from 'react';
class WelcomeMessage extends React.Component {
  render() {
```

```

    return (
      <div>
        <h2>Welcome to React!</h2>
      </div>
    );
  }
}
export default WelcomeMessage;

```

Props and State

Question 1: What are props in React.js? How are props different from state?

Answer: Props are configuration parameters passed into a component from its parent. They are strictly read-only (immutable); a component cannot modify the props it receives.

State is a private, local data store managed completely inside the component itself. State is mutable (can be changed) and is used to keep track of data that changes over time based on user interactions.

Feature	Props	State
Source	Passed by a parent component.	Managed within the component itself.
Mutable	No.	Yes.
Triggers Re-renders	Yes, when props change.	Yes, when the state changes.

Question 2: Explain the concept of state in React and how it is used to manage component data.

Answer: A state is an object that holds data specific to a component that might change over the lifecycle of the application (such as tracking if a checkbox is checked, text typed into a form, or data fetched from an API).

When state changes, React instantly detects the modification, calculates what needs to be updated in the view, and re-renders the component automatically so the UI matches the new data layer.

Question 3: Why is this.setState() used in class components, and how does it work?

Answer: In class components, you cannot update state directly (e.g., this.state.count = 5) because React won't realize the data changed, meaning the screen won't update. Instead, you must use this.setState(). It works by:

1. Taking the new data and merging it into the existing state object.
2. Telling React that a data change occurred, which automatically triggers the component's render() method to refresh the UI.

LAB EXERCISE

Task 1: Create a React component UserCard that accepts name, age, and location as props and displays them in a card format.

```

import React from 'react';

function UserCard({ name, age, location }) {
  return (
    <div style={{
      border: '1px solid #ccc',
      borderRadius: '8px',
      padding: '16px',
      width: '250px',
      boxShadow: '0 4px 6px rgba(0,0,0,0.1)',
      fontFamily: 'sans-serif',
      backgroundColor: '#f9f9f9'
    }}>
      <h3 style={{ margin: '0 0 10px 0', color: '#333' }}>{name}</h3>
      <p style={{ margin: '5px 0' }}><strong>Age:</strong> {age}</p>
      <p style={{ margin: '5px 0', color: '#666' }}><strong>Location:</strong> {location}</p>
    </div>
  );
}
export default UserCard;

```

Task 2: Create a Counter component with a button that increments a count value using React state. Display the current count on the screen.

```

import React, { useState } from 'react';
function Counter() {
  // Initialize state 'count' at 0
  const [count, setCount] = useState(0);
  return (
    <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
      <h2>Current Count: {count}</h2>
      <button
        onClick={() => setCount(count + 1)}
        style={{
          padding: '8px 16px',
          fontSize: '16px',
          backgroundColor: '#007BFF',
          color: 'white',
          border: 'none',
          borderRadius: '4px',
          cursor: 'pointer'
        }}
      >
        Increment
      </button>
    </div>
  );
}
export default Counter;
return (
  <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
    <h2>Current Count: {count}</h2>
    <button
      onClick={() => setCount(count + 1)}
      style={{

```

```
padding: '8px 16px',
fontSize: '16px',
backgroundColor: '#007BFF',
color: 'white',
border: 'none',
borderRadius: '4px',
cursor: 'pointer'
  }}
>
  Increment
</button>
</div>
);
}

export default Counter;
```